WorkHub — Software Requirements Specification (SRS)

Oct 9, 2026 · @Mohamed Saouab

1. Introduction

This SRS specifies how WorkHub v1 is built: a TypeScript Nx monorepo with a Next.js web app and a NestJS API on PostgreSQL. Product scope, personas and priorities come from the WorkHub PRD.

Scope. Multi-tenant SaaS for spaces, membership and invitations, leave policies and balances, leave requests, holidays, document requests, approvals with comments, notifications and profiles.

Definitions

Term	Meaning
Space	A tenant (company workspace); every business record belongs to exactly one space
Membership	Link between a user and a space, carrying role, team and employment data
Leave type	A category of absence with its own allowance and rules
Leave ledger	Append-only list of balance movements (accrual, deduction, adjustment, carry-over)
Request	A leave request or a document request; both share the approval and comment model
RLS	PostgreSQL row-level security

Requirement keywords. "Shall" = mandatory for v1, "should" = desirable, "may" = optional.

2. Technology decision

Decision: NestJS for the backend. With Next.js on the front and an Nx monorepo, one language end to end gives shared types, shared validation and one toolchain; FastAPI would split the repo into two ecosystems for no feature WorkHub needs.

NestJS vs FastAPI for WorkHub

Criterion	NestJS	FastAPI
Fit with Nx + Next.js	First-class Nx plugin, generators, shared libs	Works via community plugins; Python libs cannot be shared with the frontend
Shared types and validation	Zod or class-validator DTOs in a lib used by both apps	Needs OpenAPI codegen to sync TS types
Structure for a large SaaS	Modules, DI, guards, interceptors suit RBAC and multi-tenancy	Lighter; you design the structure yourself
Background jobs, queues, cron	BullMQ and @nestjs/schedule built in	Celery or ARQ, extra infra
AI / data features later	Fine via APIs (Anthropic, OpenAI SDKs)	Stronger for heavy ML or data science
Hiring market for full-stack roles	TypeScript full-stack (Next.js + Node) is one of the most requested profiles for SaaS startups and remote roles	Python backend is strong in AI and data teams

Career angle. WorkHub done in NestJS + Next.js + Nx shows a coherent senior full-stack TypeScript story: monorepo, multi-tenancy, RBAC, queues, testing, CI. Choose FastAPI only if you are targeting AI/ML or data-engineering roles specifically; then a small separate Python service (e.g. the AI assistant in a later phase) shows that skill without splitting the core.

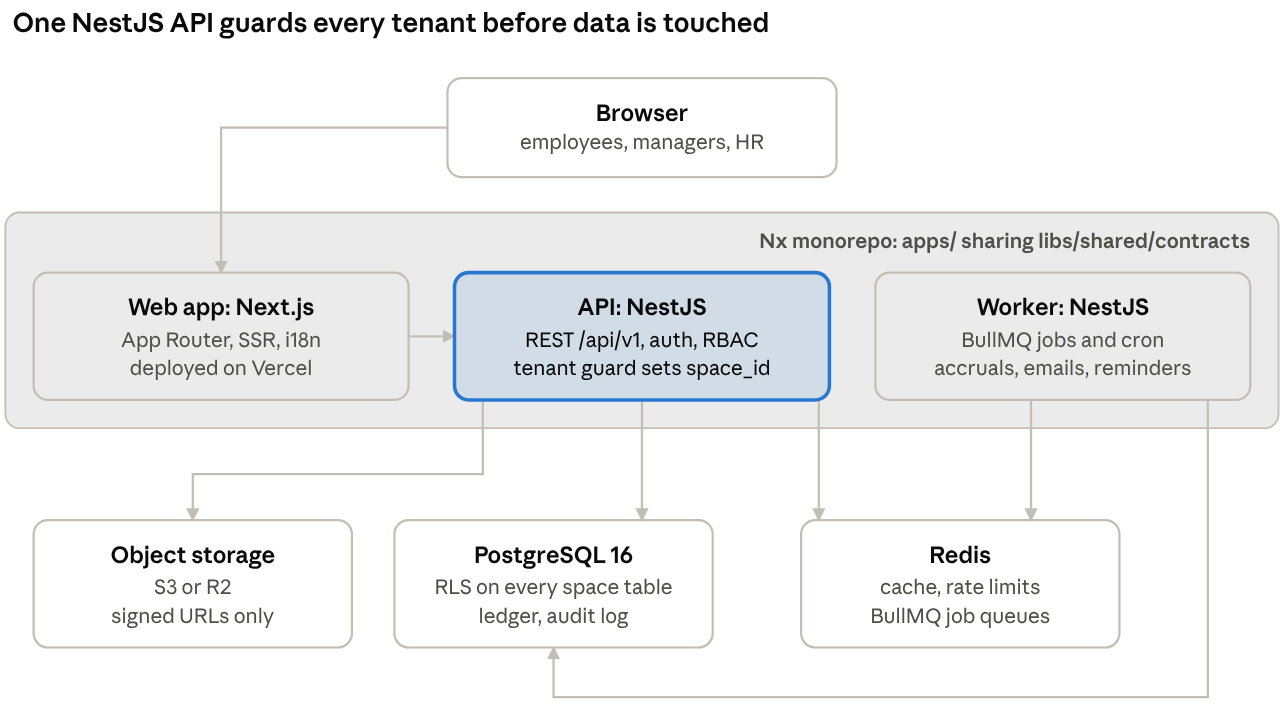
Chosen stack

Layer	Choice	Notes
Monorepo	Nx, pnpm	Affected builds, remote cache, module boundary rules

Layer	Choice	Notes
Frontend	Next.js (App Router), React, TypeScript	Server components for dashboards, client components for forms
UI	Tailwind CSS, shadcn/ui, TanStack Query, React Hook Form + Zod	RTL layout from day one via Tailwind logical properties
i18n	next-intl	EN and AR at launch; FR in v1.1
Backend	NestJS, TypeScript	REST + OpenAPI (Swagger)
ORM / DB	Prisma (or Drizzle), PostgreSQL 16	RLS for tenant isolation
Auth	Better Auth or Auth.js on Next.js + JWT verified by NestJS, or NestJS + Passport	Email/password, Google, Microsoft; magic links for invites
Cache / queue	Redis + BullMQ	Emails, accrual jobs, reminders
Files	S3-compatible storage (AWS S3, Cloudflare R2, MinIO locally)	Signed URLs only
Email	Resend or SMTP via Nodemailer, React Email templates	
Realtime	Server-Sent Events or Socket.IO gateway	In-app notifications
Observability	Pino logs, OpenTelemetry, Sentry	
Infra	Docker, GitHub Actions; Vercel for web, Railway / Fly.io / AWS ECS for API	

3. System architecture

The browser talks only to the Next.js app, which calls the NestJS API with the user's session token; the API is the single gate to data, and the worker handles everything slow or scheduled.



WorkHub runtime architecture · 3 apps, 3 data services

The API enqueues jobs in Redis that the worker consumes; both read and write PostgreSQL through the same `libs/api/database` client, so RLS applies to background jobs too. Email (Resend or SMTP) and OAuth providers (Google, Microsoft) are called from the worker and API respectively.

4. Nx monorepo structure

Two deployable apps plus a worker share typed contracts through libs; Nx tags enforce that `feature` libs never import from other apps.

```
workhub/
  apps/
    web/                # Next.js app (App Router)
    api/                # NestJS REST API
    worker/             # NestJS standalone app: BullMQ jobs, cron
(accruals, reminders)
    web-e2e/            # Playwright end-to-end tests
  libs/
    shared/
      contracts/        # Zod schemas + inferred TS types for every DTO
(used by web and api)
```

```
      constants/         # roles, permissions, enums, error codes
      utils/             # date and working-day calculation, i18n helpers
    api/
      core/              # config, logger, tenant context, guards,
interceptors
      database/          # Prisma schema, client, migrations, seed
      auth/              # auth module, session/JWT strategy
      spaces/            # spaces, memberships, invites, domains
      leaves/            # leave types, policies, ledger, requests
      holidays/
      documents/         # document types and requests, file storage
      approvals/         # decisions, comments, audit log
      notifications/     # email + in-app
    web/
      ui/                # design system (shadcn/ui components)
      data-access/       # API client, TanStack Query hooks generated from
contracts
      feature-*/         # feature screens: spaces, leaves, documents,
approvals, profile
  tools/                 # Nx generators, scripts
  docker-compose.yml     # postgres, redis, minio, mailpit
  nx.json  pnpm-workspace.yaml  tsconfig.base.json
```

Boundary rules (eslint `@nx/enforce-module-boundaries`)

- `scope:shared` may be imported by any project; it imports nothing app-specific.
- `scope:api` libs are imported only by `api` and `worker`.
- `scope:web` libs are imported only by `web`.
- `type:feature` may depend on `type:data-access`, `type:ui`, `type:util`, never the reverse.

5. Functional requirements

44 requirements across ten modules define v1; IDs are referenced by tests and tickets.

ID	Requirement
AUTH	
FR-AUTH-01	The system shall support sign-up and login with email + password, Google and Microsoft OAuth.

ID	Requirement
FR-AUTH-02	The system shall verify email addresses before a user can join a space by domain.
FR-AUTH-03	The system shall support password reset by emailed link valid 30 minutes.
FR-AUTH-04	The system should support optional TOTP two-factor authentication.
SPACES	
FR-SPC-01	A user shall create a space with name, unique slug, country, timezone, work week and logo.
FR-SPC-02	A user shall belong to multiple spaces and switch the active space; the active space is in the URL (`/s/{slug}/...`).
FR-SPC-03	On creation, the system shall seed default leave types and the country's public holidays.
FR-SPC-04	The owner shall transfer ownership, archive, or delete a space (soft delete, purge after 30 days).
MEMBERS	
FR-MEM-01	Admins shall create invite links with optional expiry and max uses, and revoke them.
FR-MEM-02	Admins shall invite by email, singly or in bulk (CSV up to 500 rows); invites expire after 7 days.
FR-MEM-03	Admins shall add a domain, verify it by DNS TXT record or admin-mailbox code, after which verified sign-ups from that domain join automatically as Employee; admins can then remove them or restrict their role.
FR-MEM-04	The system shall reject public email domains for domain join.
FR-MEM-05	Admins shall assign role, team, manager, and deactivate members; deactivated members keep history but lose access.

ID	Requirement
FR-MEM-06	Admins shall create teams and assign one or more managers per team.
LEAVE POLICY AND BANK	
FR-LVB-01	Admins shall define leave types: name, color, paid/unpaid, yearly allowance, accrual (yearly upfront or monthly), carry-over cap, requires approval, requires attachment.
FR-LVB-02	The system shall store balances as an append-only ledger; displayed balance = sum of entries.
FR-LVB-03	The worker shall run accruals and year-end carry-over per space timezone.
FR-LVB-04	Admins shall adjust a balance with a mandatory reason; adjustment is a ledger entry.
FR-LVB-05	Members shall view their own balance per type: allowance, taken, pending, remaining.
FR-LVB-06	Members shall view the team leave bank and who is off, including each teammate's exact leave type.
LEAVE REQUESTS	
FR-LVR-01	A member shall request leave: type, start, end, half-day start/end, reason, attachment.
FR-LVR-02	The system shall compute working days excluding weekends per work week and space holidays.
FR-LVR-03	The system shall block requests overlapping another pending/approved request of the same member.
FR-LVR-04	The system shall warn, and block if the type disallows negative balance, when remaining is insufficient.
FR-LVR-05	The requester shall cancel a pending request; cancelling an approved request creates a cancel request for approval.

ID	Requirement
HOLIDAYS	
FR-HOL-01	Admins shall import public holidays by country and year and add custom company holidays.
FR-HOL-02	Members shall view holidays in list and calendar views.
DOCUMENTS	
FR-DOC-01	Admins shall define document types with name, description, turnaround days and fulfilment role.
FR-DOC-02	A member shall request a document with purpose, language and delivery method.
FR-DOC-03	HR shall move a request through Pending, In progress, Ready (file uploaded) or Rejected (comment required).
FR-DOC-04	The member shall download ready documents via a signed URL valid 10 minutes.
APPROVALS AND COMMENTS	
FR-APR-01	Approvers shall see an inbox of pending requests they can decide, filterable by team, type and date.
FR-APR-02	Approvers shall approve or deny; denial shall require a comment.
FR-APR-03	A member shall not approve their own request; the owner/admin fallback approves requests of team managers.
FR-APR-04	Requester and approvers shall comment on a request thread at any status.
FR-APR-05	Every status change and comment shall be written to an immutable audit log (actor, time, from/to status).
FR-APR-06	Approval shall create ledger deductions atomically in the same transaction.

ID	Requirement
NOTIFICATIONS AND PROFILE	
FR-NTF-01	The system shall notify by email and in-app on: invite, new request (approvers), decision, comment, document ready.
FR-NTF-02	Users shall set notification preferences per event and channel.
FR-PRF-01	Members shall edit personal profile fields; admins shall edit employment fields.
FR-PRF-02	Restricted fields (national ID, salary-related) shall be visible only to the member, admins and owner.
FR-PRF-03	Members shall view their request history and export their personal data as JSON.

Billing (v1.1)

- FR-BIL-01: Billing shall be per space; the monthly price shall be computed from the number of active memberships in that space, prorated when members join or are deactivated mid-cycle.
- FR-BIL-02: Only the space owner shall manage the subscription; a user belonging to several spaces is never billed personally.

6. Data model

Every table except `user` and `session` carries `space_id`, which RLS policies match against the request's tenant context. All IDs are UUID v7; all tables have `created_at` and `updated_at`.

Entity	Key fields	Relations
user	email (unique), name, avatar_url, locale, email_verified_at	has many membership

Entity	Key fields	Relations
space	name, slug (unique), country, timezone, work_week (bitmask), logo_url, settings (jsonb), plan, billing_customer_id, deleted_at	has many membership, team, leave_type
membership	role (owner, admin, manager, employee), status (active, pending, deactivated), job_title, employee_no, hire_date, manager_membership_id	user × space (unique pair), belongs to team
team	name	space; many-to-many manager memberships via team_manager
invite_link	token_hash, expires_at, max_uses, uses, revoked_at, default_role	space, created_by
email_invite	email, token_hash, expires_at, accepted_at	space
space_domain	domain, verification_method, verified_at, join_mode (auto by default)	space
leave_type	name, color, paid, yearly_allowance, accrual (yearly, monthly), carry_over_cap, requires_approval, requires_attachment, allow_negative	space
leave_ledger_entry	kind (accrual, deduction, adjustment, carry_over, reversal), days (decimal 5,1), effective_date, reason, request_id	membership, leave_type
holiday	date, name, source (public, company), half_day	space
leave_request	start_date, end_date, start_half, end_half, working_days, reason, status	membership, leave_type, attachments

Entity	Key fields	Relations
document_type	name, description, turnaround_days, fulfilment_role	space
document_request	purpose, language, delivery, status, due_date, file_key	membership, document_type
request_comment	body, request_kind, request_id, author_membership_id	polymorphic to leave_request or document_request
audit_event	actor_membership_id, entity, entity_id, action, from_status, to_status, payload (jsonb)	space; append-only
notification	event, payload, read_at, channel	user, space
attachment	file_key, mime, size_bytes	owner request

Status enums. Leave: `pending, approved, denied, cancelled, cancel_requested`. Document: `pending, in_progress, ready, rejected, cancelled`.

7. API design

REST over HTTPS, versioned at `/api/v1`, documented with OpenAPI generated from the shared Zod contracts. Space-scoped routes live under `/spaces/{spaceId}`; a guard resolves the caller's membership and role there and sets the tenant context.

Method	Path	Purpose	Min. role
POST	/auth/sign-up, /auth/login, /auth/logout	Session management	Public
GET	/me, /me/spaces	Current user and their spaces	User
POST	/spaces	Create space	User
PATCH / DELETE	/spaces/{id}	Update / delete space	Owner

Method	Path	Purpose	Min. role
GET / POST	/spaces/{id}/members	List / add members	Employee / Admin
PATCH	/spaces/{id}/members/{mid}	Change role, team, manager, status	Admin
POST / DELETE	/spaces/{id}/invite-links	Create / revoke invite link	Admin
POST	/invites/{token}/accept	Join via link or email invite	User
POST	/spaces/{id}/domains, /domains/{did}/verify	Add and verify email domain	Admin
GET / POST	/spaces/{id}/teams	Teams	Employee / Admin
GET / POST / PATCH	/spaces/{id}/leave-types	Leave types	Employee / Admin
GET	/spaces/{id}/balances?memberId=	Leave bank (self or team, per visibility)	Employee
POST	/spaces/{id}/balances/adjustments	Manual adjustment	Admin
GET / POST	/spaces/{id}/leave-requests	List / create leave requests	Employee
POST	/spaces/{id}/leave-requests/{rid}/approve, /deny, /cancel	Decisions and cancel	Manager / Employee
GET	/spaces/{id}/calendar?from=&to=	Who is off and holidays	Employee
GET / POST	/spaces/{id}/holidays, /holidays/import	Holidays	Employee / Admin

Method	Path	Purpose	Min. role
GET / POST	/spaces/{id}/document-requests	List / create document requests	Employee
PATCH	/spaces/{id}/document-requests/{rid}	Status change, file upload	Admin (HR)
GET / POST	/spaces/{id}/requests/{kind}/{rid}/comments	Comment thread	Requester / Approver
GET	/spaces/{id}/approvals/inbox	Pending items for approver	Manager
GET / PATCH	/notifications, /notifications/{nid}/read	In-app notifications	User

Conventions. Cursor pagination (`?cursor=&limit=` , max 100); errors as RFC 9457 problem JSON with a stable `code` ; `Idempotency-Key` header on create endpoints; optimistic locking via `version` on request decisions to prevent double approval.

8. Non-functional requirements

Tenant isolation is the top non-functional requirement: a bug in one query must never expose another company's data.

ID	Area	Requirement
NFR-01	Tenant isolation	Every space-scoped query shall run with `app.space_id` set per transaction and PostgreSQL RLS enabled on all tenant tables; integration tests shall prove cross-space reads return nothing.
NFR-02	Authorization	RBAC shall be enforced server-side in NestJS guards with a permission map in `libs/shared/constants` ; the UI only hides actions.
NFR-03	Performance	p95 API latency < 300 ms for reads and < 500 ms for writes at 50 requests/s; dashboard first load < 2.5 s LCP on 4G.

ID	Area	Requirement
NFR-04	Scalability	API and worker shall be stateless and horizontally scalable; target 1,000 spaces and 50,000 members on one Postgres instance.
NFR-05	Availability	99.5% monthly uptime; daily backups with 14-day retention and point-in-time recovery; RPO 1 h, RTO 4 h.
NFR-06	Security	OWASP ASVS level 2: argon2id password hashing, httpOnly SameSite cookies, CSRF protection, rate limiting (login 5/min/IP), Helmet headers, input validation on every DTO, secrets in a vault, dependency scanning.
NFR-07	Files	Uploads limited to PDF, PNG, JPG up to 10 MB, virus-scanned, stored private, served by signed URL.
NFR-08	Privacy	Compliant with Morocco Law 09-08 and GDPR: data export, account deletion, restricted fields encrypted at rest, data processing records, EU hosting by default.
NFR-09	Audit	Audit events shall be append-only and retained 5 years.
NFR-10	i18n	All UI strings externalised; EN and AR at launch, with full RTL (mirrored layout, Arabic fonts, bidirectional text); FR in v1.1; dates and numbers formatted per locale and space timezone.
NFR-11	Accessibility	WCAG 2.2 AA for all core flows; keyboard-navigable approval inbox.
NFR-12	Observability	Structured logs with request ID and space ID, traces via OpenTelemetry, error tracking in Sentry, alert on 5xx > 1% over 5 min.
NFR-13	Maintainability	Strict TypeScript, ESLint, Prettier, Nx module boundaries; 80% unit coverage on `libs/api/*` domain logic.

ID	Area	Requirement
NFR-14	Compatibility	Latest 2 versions of Chrome, Edge, Firefox, Safari; responsive from 360 px wide.

9. Testing, CI/CD and deployment

Every pull request runs only the affected projects through lint, type-check, tests and build; `main` deploys to staging automatically and to production on a tagged release.

Test strategy

Level	Tool	Covers
Unit	Jest (api, libs), Vitest (web)	Working-day calculation, accrual, ledger math, permission map
Integration	Jest + Testcontainers (Postgres, Redis)	Repositories, RLS isolation, approval transactions
API contract	Supertest against OpenAPI	Status codes, validation, auth per role
E2E	Playwright	Create space → invite → request leave → approve → balance updated; document request to download
Accessibility	axe-core in Playwright	Core flows

Pipeline (GitHub Actions)

1. `pnpm install` with Nx remote cache (Nx Cloud or self-hosted).
2. `nx affected -t lint typecheck test build`.
3. Integration tests against service containers.
4. Build Docker images for `api` and `worker`; Vercel preview deploy for `web`.
5. Run Prisma migrations on staging, then Playwright E2E against staging.
6. On release tag: migrate production, deploy images, promote web.

Environments. Local (docker-compose: Postgres, Redis, MinIO, Mailpit), preview per PR, staging, production. Config via environment variables validated at boot with Zod.

Deployment target (v1). `web` on Vercel; `api` and `worker` as containers on Railway, Fly.io or AWS ECS; managed PostgreSQL (Neon, Supabase or RDS) and Redis (Upstash or ElastiCache); Cloudflare R2 for files.